

Freshness Ads

SDK quảng cáo AdMob cho Android — waterfall theo tier, native pool, consent UMP, cấu hình bằng Remote Config.

Phiên bản 1.0.0 · Google Mobile Ads Next-Gen SDK 1.2.1 · minSdk 26 · Hướng dẫn sử dụng

Nội dung

1. SDK này giải quyết chuyện gì
2. Cài đặt
3. File cấu hình `ads_id_config.json`
4. Khởi tạo trong Application
5. Consent (UMP) và luồng splash
6. Interstitial
7. Native — một slot
8. Native — pool cho danh sách
9. Native full màn
10. Banner
11. Rewarded
12. App-open
13. Remote Config
14. ProGuard / R8
15. Mediation Unity Ads
16. Test và debug
17. Những lỗi làm tụt show rate

1. SDK này giải quyết chuyện gì

Gọi thẳng Google Mobile Ads SDK thì mỗi màn hình phải tự lo: load, cache, huỷ ad đúng lúc, chờ consent xong mới request, xử lý no-fill, đổi ad unit id khi cần. Làm lại từng đó ở mỗi app là chỗ sinh bug và sinh lãng phí request.

SDK này gom lại thành bốn thứ:

Thứ	Nghĩa là gì
Catalog	Ad unit id không nằm trong code. Chúng nằm trong <code>assets/ads_id_config.json</code> , và Remote Config ghi đè được từng placement — đổi id, tắt một chỗ đặt ad, thiết thời gian chờ, tắt cả không cần release.
Waterfall	Mỗi placement khai được nhiều id xếp theo thứ tự. Id đầu không fill thì tự xuống id kế. Dùng để đặt tầng high-floor phía trên.
Ngân sách thời gian	Mỗi format có deadline riêng, và deadline đó chia cho từng tier. Hết ngân sách thì dừng, không để user ngồi chờ một request không bao giờ về.
Vòng đời	Ad được cache, được huỷ đúng lúc view chết, và fill về muộn vẫn được giữ cho lần load sau thay vì vứt đi.

Một đầu vào duy nhất: `AdsGraph.adsManager`. Không có Hilt/Dagger, không cần kapt/KSP.

2. Cài đặt

2.1 Thêm repository và dependency

Trong `settings.gradle.kts`:

```
dependencyResolutionManagement {
    repositories {
        google()
        mavenCentral()
        maven { url = uri("https://jitpack.io") }
    }
}
```

Trong `app/build.gradle.kts`:

```
dependencies {
    implementation("com.github.vuhuyvqh2112:freshness-ads:1.0.0")
}
```

Không cần khai thêm GMA SDK, UMP, Unity adapter hay Firebase Config — POM của SDK đã mang theo hết.

2.2 Firebase

SDK đọc Remote Config và ghi non-fatal vào Crashlytics, nên app **bắt buộc** phải có Firebase:

```
// build.gradle.kts (root)
plugins {
    id("com.google.gms.google-services") version "4.5.0" apply false
    id("com.google.firebase.crashlytics") version "3.0.7" apply false
}

// app/build.gradle.kts
plugins {
    id("com.google.gms.google-services")
    id("com.google.firebase.crashlytics")
}
```

Đặt `google-services.json` vào thư mục `app/` .

2.3 AdMob app id trong manifest

```
<application>
    <meta-data
        android:name="com.google.android.gms.ads.APPLICATION_ID"
        android:value="ca-app-pub-XXXXXXXXXXXXXXXX~YYYYYYYYYY" />
</application>
```

Đây là APP ID, không phải ad unit id. App id có dấu `~` , ad unit id có dấu `/` . Điền nhầm là app crash ngay lúc khởi động, không phải "không có ad".

Quyền `INTERNET` , `AD_ID` và `install-referrer` đã nằm trong manifest của SDK, không phải khai lại.

3. File cấu hình `ads_id_config.json`

Chép `samples/ads_id_config.json` vào `app/src/main/assets/ads_id_config.json`. Đây là bản mặc định đóng gói trong APK, dùng khi Remote Config chưa fetch về kịp (lần mở app đầu tiên, hoặc mất mạng).

```
{
  "enableAllAds": true,
  "settings": {
    "splashTimeoutMs": 30000,
    "interMinIntervalMs": 20000,
    "rewardTimeoutMs": 20000
  },
  "placements": {
    "inter_splash": {
      "format": "interstitial",
      "enable": true,
      "ids": [
        { "id": "ca-app-pub-xxx/high-floor", "enable": true },
        { "id": "ca-app-pub-xxx/all-price", "enable": true }
      ]
    },
    "native_home": {
      "format": "native",
      "enable": true,
      "baseMs": 30000,
      "tierCapMs": 8000,
      "ceilingMs": 120000,
      "ids": [
        { "id": "ca-app-pub-xxx/native-home", "enable": true }
      ]
    }
  }
}
```

Trường	Ý nghĩa
<code>enableAllAds</code>	Công tắc tổng. <code>false</code> là tắt sạch mọi format.
<code>placements.<key></code>	Tên chỗ đặt ad. Do bạn tự đặt — SDK không ép danh sách nào.
<code>format</code>	<code>banner</code> · <code>native</code> · <code>interstitial</code> · <code>rewarded</code> · <code>appOpen</code> . Quyết định test id dùng ở debug build.
<code>enable</code> (placement)	<code>false</code> = tắt cả chỗ đặt ad này.
<code>ids</code>	Thứ tự waterfall. Id đầu thử trước; fail mới xuống id kế. Tầng high-floor chèn vào đầu mảng.
<code>enable</code> (id)	<code>false</code> = tắt riêng một tầng, giữ nguyên các tầng khác.
<code>baseMs</code> / <code>tierCapMs</code> / <code>ceilingMs</code>	Ghi đè ngân sách thời gian mặc định của format. Bỏ trống là dùng mặc định.

Placement không tồn tại = tắt. Key không có ở cả Remote Config lẫn file assets sẽ resolve ra rỗng, và mọi lệnh load cho nó im lặng bỏ qua. Không crash, nhưng cũng không có ad — gõ sai tên placement là lỗi

khó thấy nhất khi tích hợp.

Hai placement SDK tự gọi

Hầu hết placement được truyền vào từ call site. Riêng hai cái này SDK tự gọi nên phải khai tên trong `AdsConfig` (mặc định `open_all` và `inter_splash`):

Placement	Vì sao SDK phải biết sẵn
<code>open_all</code>	App-open ad load ngầm theo vòng đời process — không có màn hình nào truyền key vào.
<code>inter_splash</code>	Interstitial của splash có ngân sách thời gian riêng (không giãn theo số tier), nên SDK phải nhận ra nó giữa các interstitial khác.

4. Khởi tạo trong Application

```
class MyApp : Application() {  
  
    override fun onCreate() {  
        super.onCreate()  
  
        AdsGraph.install(  
            application = this,  
            config = AdsConfig(  
                openAdPlacement = "open_all",  
                splashInterstitialPlacement = "inter_splash",  
                // key placement -> số ad giữ sẵn trong pool  
                nativePools = mapOf("native_home" to 1),  
            ),  
        )  
  
        // App-open ad phải được che bằng một Activity đực trước khi hiện  
        // (yêu cầu policy của AdMob: user không được thấy nội dung app sau lưng ad).  
        AdsGraph.adsManager.setOnShowOpenAdListener {  
            AdsGraph.adsManager.topActivity.get()?.let { OpenAdActivity.start(it) }  
        }  
    }  
}
```

`install` gọi một lần; lần thứ hai là no-op. Sau đó dùng `AdsGraph.adsManager` ở mọi nơi.

Nối với IAP

Mặc định chỉ `enableAllAds` quyết định bật/tắt. App có mua-gói-bỏ-quảng-cáo thì truyền `AdsDataStore` riêng:

```
AdsGraph.install(  
    application = this,  
    config = AdsConfig(/* ... */),  
    dataStore = object : AdsDataStore {  
        override var isPurchased: Boolean = billing.hasRemoveAds  
        override val isAdEnabled: Boolean  
            get() = !isPurchased && remoteFlags.adsOn  
    },  
)
```

5. Consent (UMP) và luồng splash

`launchSplashAdFlow` làm trọn gói: gom consent (có timeout), rồi hiện interstitial splash nếu placement đó đang bật.

```

class SplashActivity : AppCompatActivity() {

    private val ads get() = AdsGraph.adsManager

    private val splashInterConfig = InterAdConfig(
        placement = "inter_splash",
        reload = false,
        timeOut = 30_000L,
        // Một lần một unit, không retry: waterfall tuần tự nhanh hơn. Retry làm id
        // chính fill muộn, tới lúc đó user đã rời splash – matched nhưng không show.
        retryCount = 1,
    )

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Bắn request NGAY, song song với consent và Remote Config.
        if (ads.isAdEnabled("inter_splash")) ads.loadInterAd(splashInterConfig)

        ads.setIsSplash(true)
        lifecycleScope.launch {
            val shown = try {
                ads.launchSplashAdFlow(splashInterConfig)
            } finally {
                // finally, KHÔNG phải dòng sau lệnh suspend: user bấm back giữa lúc chờ
                // là scope bị huỷ và cờ splash kết true, chặn mọi app-open ad về sau.
                ads.setIsSplash(false)
            }
            goToNextScreen()
        }
    }
}

```

Bắt buộc cho user EEA/UK. Policy của Google yêu cầu app có lối vào để đổi/rút lại consent. Kiểm tra `ads.isPrivacyOptionsRequired` rồi hiện một mục trong Settings gọi `ads.showPrivacyOptionsForm(activity)`. Thiếu cái này là rủi ro bị gỡ app.

6. Interstitial

```
private val interConfig = InterAdConfig(
    placement = "inter_next_screen",
    reload = true,          // tự load lại sau khi hiện xong
    retryCount = 2,
    timeOut = 6_000L,
    isShowLoading = true, // hiện dialog loading trong lúc chờ fill
)

// Warm sớm, ở màn trước đó
ads.loadInterAd(interConfig)

// Lúc chuyển màn
lifecycleScope.launch {
    ads.showInterAd(interConfig, placement = "inter_next_screen")
    startActivity(Intent(this@Screen, NextActivity::class.java))
}
```

`showInterAd` trả về sau khi user đóng ad, và trả `false` nếu không có gì để hiện. Luôn đi tiếp bất kể kết quả — đừng chặn điều hướng vì một quảng cáo không fill.

SDK tự áp **frequency cap** giữa hai interstitial bất kỳ (mặc định 20s, chỉnh bằng `settings.interMinIntervalMs`). Bấm nhanh qua nhiều màn sẽ không ăn liên tiếp hai quảng cáo toàn màn.

7. Native — một slot

Đặt view vào layout. Chọn template theo chỗ đặt:

```
<com.freshness.ads.natives.LoadingNativeAdView1
    android:id="@+id/nativeAd"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Class	Dùng cho
<code>LoadingNativeAdView1 ... View4</code>	Bốn bố cục chuẩn, khác nhau ở vị trí media và nút CTA
<code>LoadingNativeAdSmallView</code>	Dải nhỏ, không có ảnh lớn
<code>LoadingNativeAdHomeView / HomeCompactView</code>	Khối trên màn chính
<code>LoadingNativeAdListView / CategoryView</code>	Item chèn giữa danh sách
<code>LoadingNativeCtaTopView / CtaBottomView</code>	CTA nằm trên / dưới
<code>LoadingNativeAdDisplayView</code>	Khối lớn, nhiều ảnh
<code>LoadingNativeFullscreenView</code>	Native chiếm trọn màn

Tiền tố `Loading` nghĩa là view đã kèm shimmer: nó tự hiện khung xám trong lúc chờ và tự đổi sang ad khi có.

Trong Activity/Fragment:

```
private fun showNativeAd() {
    if (!ads.isAdEnabled("native_detail")) {
        binding.nativeAd.isVisible = false
        return
    }
    val service = ads.nativeAdService
    service.preloadIfEmpty(applicationContext, key = "native_detail", placement =
"native_detail")
    bindNativeAdFromCache(
        container = binding.nativeAd,
        nativeAdService = service,
        cacheKey = "native_detail",
        // true cho màn một-lần: ad được huỷ khi màn chết, NHƯNG chỉ khi nó đã thật
        // sự ăn impression. Fill chưa kịp hiện thì giữ lại cho lần vào sau.
        releaseOnDestroy = true,
    )
}
```

Hai khái niệm khác nhau: key và placement .

placement quyết định *load id* nào (key trong `ads_id_config`).

key định danh *chỗ hiển thị* trong cache. Hai màn dùng chung một placement vẫn phải có key riêng, nếu không màn thứ hai sẽ bind đúng cái ad màn thứ nhất đang giữ.

Preload sớm một màn

`preloadIfEmpty` là idempotent: gọi lại khi đã có ad hoặc đang load thì không phát thêm request. Nhờ vậy gọi được từ màn trước để ad sẵn sàng trước khi user tới:

```
// Ở màn A, hàm sẵn native của màn B
ads.nativeAdService.preloadIfEmpty(applicationContext, "native_b", "native_b")
```

8. Native — pool cho danh sách

Danh sách chèn nhiều vị trí ad thì dùng `AdPoolManager` : nó preload sẵn một số ad và phát ngay khi adapter cần, thay vì load trong `onBindViewHolder` (vừa giật vừa phát dư request).

```
// Khai pool lúc install
AdsGraph.install(
    application = this,
    config = AdsConfig(nativePools = mapOf("native_feed" to 3)),
)

// Hàm khi mở màn
val pool = ads.adPoolManager
pool.preloadIfEmpty("native_feed")

// Trong adapter
val ad = pool.getAd("native_feed", key = "native_feed_$position", owner = holder)
if (ad != null) {
    holder.adView.setNativeAd(ad)
} else if (pool.isPlacementExhausted("native_feed")) {
    holder.adView.isVisible = false // hết ad thật, thu gọn slot
}                                     // còn lại: để shimmer chạy tiếp

// Khi holder bị recycle
pool.releaseAd("native_feed", key = "native_feed_$position", owner = holder)
```

Muốn bind lại ngay khi có ad mới về, lắng nghe `pool.adReadyEvents` :

```
lifecycleScope.launch {
    repeatOnLifecycle(Lifecycle.State.STARTED) {
        pool.adReadyEvents.collect { placement ->
            if (placement == "native_feed") adapter.notifyAdSlotsChanged()
        }
    }
}
```

Pool size phải bằng số slot hiển thị CÙNG LÚC. Màn chỉ có một slot tĩnh mà khai 2 thì ad thứ hai không có đường ra: slot đã có ad rồi thì lần bind sau bỏ qua. Kết quả là mỗi phiên phát dư một request không bao giờ đổi được impression — show rate tụt thẳng. Chỉ tăng pool khi có feed thật sự hiển thị nhiều ad song song.

`owner` là tham số quan trọng: RecyclerView có thể tạo holder mới cho cùng một vị trí *trước khi* holder cũ bị recycle (ItemAnimator còn dựng hẵn holder thứ hai để crossfade). Truyền `owner` để pool chỉ huỷ ad khi không còn holder nào giữ nó. Không truyền thì holder cũ sẽ huỷ mất ad mà holder mới đang hiển thị, và frame sau crash *"trying to use a recycled bitmap"*.

9. Native full màn

Native chiếm trọn màn hình dùng làm fallback khi interstitial không fill, hoặc chèn giữa các bước onboarding. Dựng bằng một Activity riêng:

```

class NativeFullscreenActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityNativeFullBinding.inflate(layoutInflater)
        setContentView(binding.root)

        val service = AdsGraph.adsManager.nativeAdService
        // Fill có thể biến mất giữa lúc màn gọi kiểm tra và lúc màn này dựng xong.
        if (!service.isReady("native_full")) { finish(); return }

        bindNativeAdFromCache(
            container = binding.nativeFull,
            nativeAdService = service,
            cacheKey = "native_full",
            releaseOnDestroy = true,
        )
        binding.btnClose.setOnClickListener { finish() }
    }
}

```

Luôn phải có đường thoát. Nút đóng phải hiện, và BACK phải chạy được. Muốn giữ impression đủ lâu để tính thì hoãn nút đóng vài giây bằng một `CountDownTimer` — nhưng đừng chặn BACK, và đừng để màn hình nào không thoát được.

10. Banner

```

<FrameLayout
    android:id="@+id/bannerContainer"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />

```

```

ads.bannerAdService.loadBannerAds(
    context = this,
    bannerView = binding.bannerContainer,
    placement = "banner_home",
    isCollapsible = false,
    onLoadSuccess = { binding.bannerContainer.isVisible = true },
    onLoadFail = { binding.bannerContainer.isVisible = false },
)

```

`isCollapsible = true` cho banner thu gọn được (user bấm mũi tên để xếp lại). Chỉ dùng ở banner neo đáy, và đừng dùng ở màn mà user quay lại liên tục — banner collapsible bung ra mỗi lần load là gây khó chịu.

11. Rewarded

```
private val rewardConfig = RewardAdConfig(
    placement = "reward_unlock",
    retryCount = 2,
    timeOut = 20_000L,
    isShowLoading = true,
)

ads.loadRewardAd(rewardConfig) // warm trước

lifecycleScope.launch {
    var earned = false
    val shown = ads.showRewardAd(
        config = rewardConfig,
        placement = "reward_unlock",
        onShow = { /* quảng cáo đã hiện toàn màn - tắt spinner của bạn ở đây */ },
        onReward = { earned = true },
    )
    if (shown && earned) unlockFeature()
    else showCouldNotLoadMessage()
}
```

Đừng chờ giá trị trả về để tắt spinner. `showRewardAd` chỉ trả về sau khi user đóng quảng cáo. Call site nào đang giữ spinner chờ fill mà đợi giá trị trả về thì spinner nằm dưới quảng cáo suốt cả video rồi loé lên một nhíp lúc đóng. Dùng `onShow`.

Và: hỏi quyền / kiểm điều kiện **trước** khi mở quảng cáo. Bắt user xem hết video rồi mới báo "thiếu quyền" là lừa họ bỏ công lấy một thứ không nhận được.

12. App-open

App-open ad hiện khi user quay lại app từ background. SDK tự theo dõi vòng đời process; app chỉ cần cung cấp một Activity để che nội dung:

```
// Application.onCreate
AdsGraph.adsManager.setOnShowOpenAdListener {
    AdsGraph.adsManager.topActivity.get()?.let { OpenAdActivity.start(it) }
}

// OpenAdActivity: theme trong suốt/đọc, không layout
class OpenAdActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        onBackPressedDispatcher.addCallback(this) { /* chặn back khi ad đang chạy */ }
        lifecycleScope.launch {

            AdsGraph.adsManager.openAdService.showAdIfAvailable(WeakReference(this@OpenAdActivity))
                finish() // đóng dù có ad hay không, đừng để lại màn trắng
                overridePendingTransition(0, 0)
        }
    }
}
```

Tạm tắt app-open cho một lần quay lại (ví dụ user vừa đi ra ngoài chọn ảnh rồi quay vào):

```
ads.setSkipOpenAd(true)
```

Bắt buộc: che nội dung app. Policy AdMob không cho user thấy nội dung app phía sau app-open ad. Không có Activity che là vi phạm.

13. Remote Config

SDK đọc **đúng một** key Firebase Remote Config: `ads_id_config`. Giá trị là toàn bộ JSON ở mục 3.

Ghi đè theo **từng placement**: key nào có trên Remote Config thì bản đó thắng trọn vẹn, key nào thiếu thì rơi về file assets. Nhờ vậy đẩy lên RC một placement duy nhất là chỉnh được riêng nó.

```
// Đọc settings
val timeout = ads.remoteConfig.splashTimeoutMs ?: 30_000L

// Kiểm công tắc một chỗ đặt ad
if (ads.isAdEnabled("native_home")) { /* ... */ }
```

Payload hỏng không làm mất ads. JSON rỗng, null hoặc parse lỗi đều bị bỏ qua và catalog giữ nguyên bản đang dùng. SDK không bao giờ tự đẩy mình về trạng thái rỗng — đó là cách nhanh nhất để mất sạch doanh thu vì một dấu phẩy thừa.

14. ProGuard / R8

Rule của SDK đi kèm trong AAR (`consumer-rules.pro`), tự áp vào app khi minify. Không phải chép gì.

Rule đã bao gồm: model JSON cấu hình, native ad view inflate từ XML, GMA next-gen + mediation adapter, Unity Ads, và constructor Room mà WorkManager gọi bằng reflection.

Nếu release crash ngay khi mở app với `Failed to create an instance of androidx.work.impl.WorkDatabase` : WorkManager đi kèm GMA và Unity Ads, nó dựng một Room database từ `androidx.startup` — trước cả `Application` . Rule này đã có sẵn trong SDK; chỉ khi bạn tự tắt consumer rules mới cần thêm tay:

```
-keep class * extends androidx.room.RoomDatabase { <init>(); }
```

15. Mediation Unity Ads

Adapter Unity đã nằm trong SDK. Không cần khai gì trong manifest, không cần code init — adapter tự nhận cấu hình từ mediation group phía AdMob lúc chạy.

Phải tạo mediation group trên AdMob TRƯỚC khi phát hành. Chưa có group thì adapter nhận cấu hình rỗng, nó init bằng một app id giả và *giữ luôn callback init của GMA*. Đo thực tế: `MobileAds.initialize` mất ~30 giây thay vì ~2,2 giây — đủ để mọi request đầu phiên thấy "SDK not ready" và interstitial splash không bao giờ kịp hiện.

Triệu chứng nhìn giống hết lỗi code, nhưng nguyên nhân nằm ở cấu hình phía AdMob.

Version của adapter và của Unity SDK phải đi thành cặp: ba số đầu của adapter khớp version SDK nó bọc (adapter `4.18.0.x` bọc `unity-ads 4.18.0`). SDK này ghim cặp `4.18` vì đó là cấu hình đã chạy production cùng GMA next-gen `1.2.1` . Đừng nâng lên bản mới hơn chỉ vì nó mới hơn — adapter mediation phải khớp với GMA next-gen, và chuyện đó chỉ kiểm được bằng một app đang sống.

16. Test và debug

Debug build **mặc định ép test id của Google**, bắt kể id thật trong config. Nhờ vậy dev không bao giờ vô tình bấm vào quảng cáo thật (dẫn tới khoá tài khoản AdMob).

Muốn debug dùng id thật + waterfall thật:

```
AdsGraph.adUnitCatalog.useRealIds = true
```

Bắt buộc phải bật khi cần kiểm hành vi rớt tier: với test id thì tier nào cũng fill, và waterfall không bao giờ rớt xuống tier sau. Release build bỏ qua hoàn toàn cờ này.

Log: SDK dùng Timber và chỉ plant cây log ở build debuggable, nên release không in gì. Lọc theo tag:

```
adb logcat -s NativeAds AdPool OpenAdProvider AdsManager AdsInitializer
```

17. Những lỗi làm tụt show rate

Show rate = impression ÷ matched request. Con số thấp nghĩa là bạn xin ad về rồi không dùng — và AdMob nhìn thấy điều đó.

Triệu chứng	Nguyên nhân thường gặp	Cách sửa
Một placement chỉ vài phần trăm	Preload ở mọi phiên nhưng chỉ hiển thị ở một nhánh hiếm ai đi vào	Hoặc thôi preload nó, hoặc tìm chỗ hiển nó ở luồng chính
Đúng khoảng 50%	Pool size 2 mà màn chỉ có một slot	Đặt pool size bằng số slot hiển thị cùng lúc
Native cứ vào lại màn là tụt	Hủy ad khi màn chết dù nó chưa từng hiện	Dùng <code>releaseOnDestroy = true</code> — SDK chỉ hủy ad đã ăn impression, fill chưa hiện được giữ lại cho lần sau
App-open thấp kinh niên	Bản chất format: luôn phải ôm sẵn một ad cho lần quay lại có thể không bao giờ tới	Trần lý thuyết là $R/(R+1)$ với R là số lần quay lại mỗi phiên. Đừng đánh đổi impression để kéo con số này lên
Interstitial splash matched mà không show	<code>retryCount</code> lớn làm id chính fill muộn, tới lúc đó user đã rời splash	Đặt <code>retryCount = 1</code> , để waterfall tuần tự xuống tier sau

Nguyên tắc gọn

- Chỉ preload thứ bạn thật sự sẽ hiển thị, ở luồng người ta thật sự đi qua.
- Pool size = số slot nhìn thấy cùng lúc. Không hơn.
- Fill chưa hiện là tiền chưa tiêu — giữ lại, đừng hủy.
- Ad không fill thì thu gọn slot, đừng để shimmer chạy mãi.
- Đừng chặn điều hướng vì một quảng cáo. Ad không về thì cho user đi tiếp.